

Internet Programming I

Chapter 4

JavaScript: Regular Expression and Form Validation



Group: 2
3rd Year Sec B Students,
Dept. of Software Eng.

Jan 2022, AASTU

Objectives



After success completion of the session you will be able to:

- Differentiate client side and server scripting language.
- know the methods of regular expression
- validate forms on the client side

Regular expression

- creating a regular expression
- writing a regular expression patterns
- character class or (brackets)
- metacharacters
- quantifiers
- methods of regular expression
- flags

Form validation

1. Creating a regular expression

- Regular expressions are patterns used to match character combinations in strings.
- In JavaScript, regular expressions are also objects.
- These patterns are used with the `exec()` and `test()` method of `RegExp`.
- And with the `match()`, `matchAll()`, `replace()`, `replaceAll()`, `search()`, and `split()` methods of string.
- which means we can use the regular expression to match, replace, search, split.
- you can construct a regular expression on two ways
 - Using a regular expression literal, which consists of a pattern enclosed between slashes, as follows:

```
let re = /ab+c/;
```

making the regular expression constant will improve performance because regular expression literals provide compilation of the regular expression when the script is loaded,

cont'd

- The second way is by calling the constructor function of the RegExp object as follow:

```
let re = new RegExp('ab+c');
```

- Using the function provide run time compilation.

Writing a regular expression pattern

- Any character inside the two forward slashes is a regular expression.

Using simple patterns

- Simple patterns are constructed of characters for which you want to find a direct match.

Using special characters

- When the search for a match requires something more than a direct match, such as finding one or more b's, or finding white space, you can include special characters in the pattern.

Escaping

- If you need to use any of the special characters literally (actually searching for a "*", for instance), you must escape it by putting a backslash in front of it.
- Using parentheses around any part of the regular expression pattern causes that part of the matched substring to be remembered.

Character class or (brackets)

Expression	Description
[abc]	find any character between the brackets
[^abc]	find any character NOT btween the brackets
[0-9]	find any character between the brackets(any digit)
[^0-9]	find any character NOT between the brackets(any non-digit)
(x)	A grouping or subpatter, which is also strored for later use
(x y)	find any of the alternatives specified

Quantifiers

Qunatifier	Description
*	match zero or more times.
+	match one or more times.
?	match zero or one time.
{n}	match exactly n times.
{n,}	match at least n times.
{n,m}	match from n to m times.

Metacharacter

Metacharacter	Description
.	find a single character, except newline or line terminator
\w	find a word character
\W	find a non-word character
\d	find a digit
\D	find a non-digit character
\s	find a whitespace character
\S	find a non-whitespace character
\0	find a NULL character
\n	find a new line character

Methods of regular expression

Method	Description
<code>exec()</code>	Executes a search for a match in a string. It returns an array of information or null on a mismatch.
<code>test()</code>	Tests for a match in a string. It returns true or false.
<code>match()</code>	Returns an array containing all of the matches, including capturing groups, or null if no match is found.
<code>matchAll()</code>	Returns an iterator containing all of the matches, including capturing groups.
<code>replaceAll()</code>	Executes a search for all matches in a string, and replaces the matched substrings with a replacement substring.
<code>split()</code>	Uses a regular expression or a fixed string to break a string into an array of substrings.
<code>search()</code>	Tests for a match in a string. It returns the index of the match, or -1 if the search fails.
<code>replace()</code>	Executes a search for a match in a string, and replaces the matched substring with a replacement substring.

Flags

Flag	Description
d	generate indices for substing matches.
g	global search
i	case-insensitive search
m	multi-line search
s	allow . to mach newline characters.
u	“unicode”; treat a pattern as a sequence of unicode code points.

Form validation

- When you enter data, the browser and/or the web server will check to see that the data is in the correct format and within the constraints set by the application.
- Validation done in the browser is called client-side validation, while validation done on the server is called server-side validation.
- There are three main reasons for insisting form validation
 - We want to get the right data, in the right format.
 - We want to protect our users' data.
 - We want to protect ourselves.

Types of form validation

- There are two different types of **client-side validation**

HTML5 form validation

uses HTML5 form validation features

JavaScript

- validation is coded using JavaScript. This validation is completely customizable, but you need to create it all (or use a library).
- JavaScript provides facility to validate the form on the client-side so data processing will be faster than server-side validation.
- Through JavaScript, we can validate name, password, email, date, mobile numbers and more fields.

server side form validation

- PHP can validate form input server side, submitted by the user using HTML forms.

Form validation using regular expression

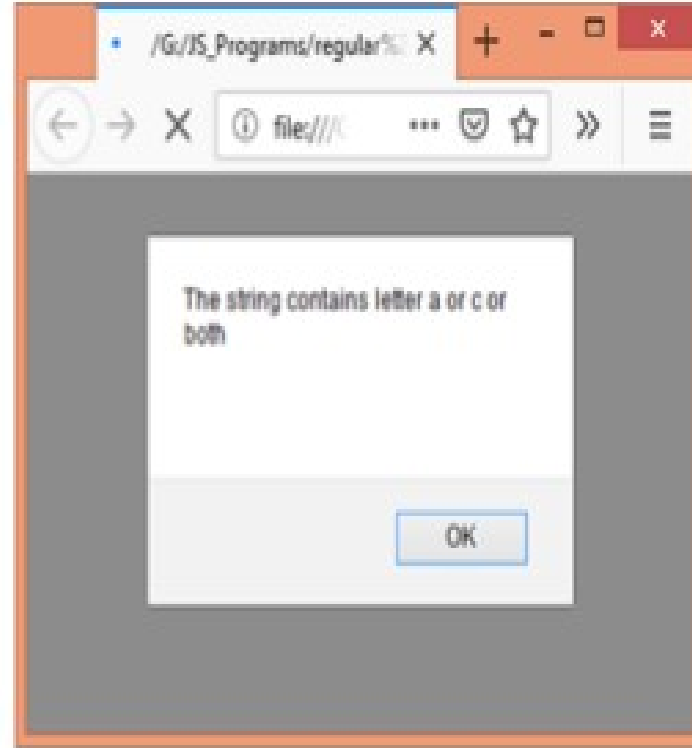
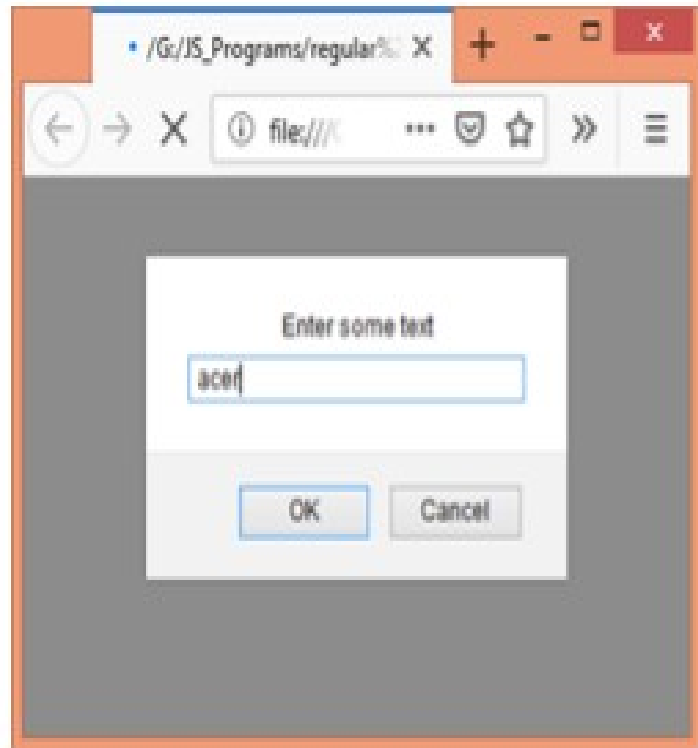
➤ WE CAN USE REGULAR EXPRESSION IN FORM FOR DIFFERENT PURPOSE WITH DIFFERENT STYLE AND METHOD, SUCH AS:-

- Regex to Check for Valid Username
- Regex to Check for Valid Phone Numbers
- Regex to Check for Valid Email Address
- Regex to Check for a Valid URL
- Matching a Password
- Matching an HTML Tag

Sample program

```
<html>
<body>
<script language="javascript">
  var input = prompt('Enter some text')
  re = /[ac]/
  if(re.test(input)) {
    alert('The string contains letter a or c or both')
  }
  else {
    alert('String does not contain a or c or both')
  }
</script>
</body>
</html>
```

Sample program output

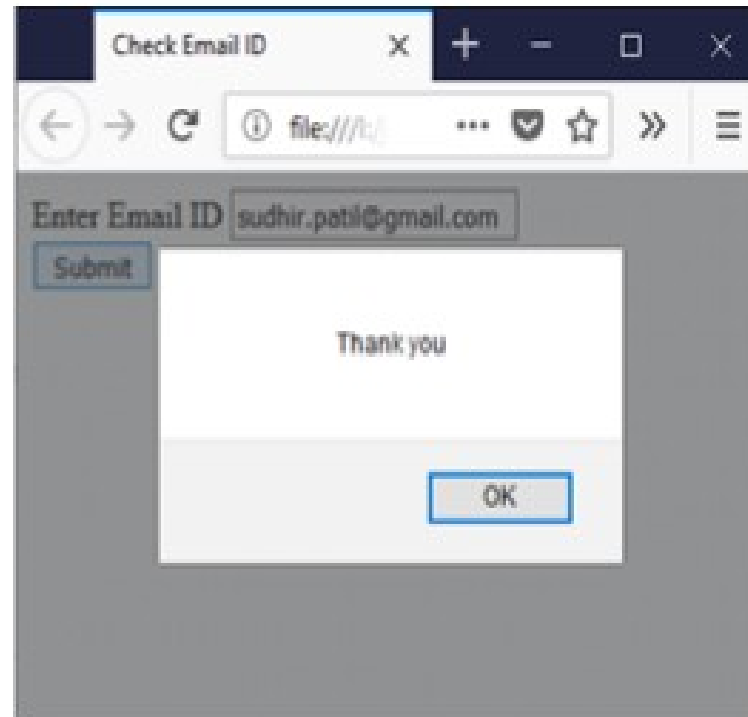
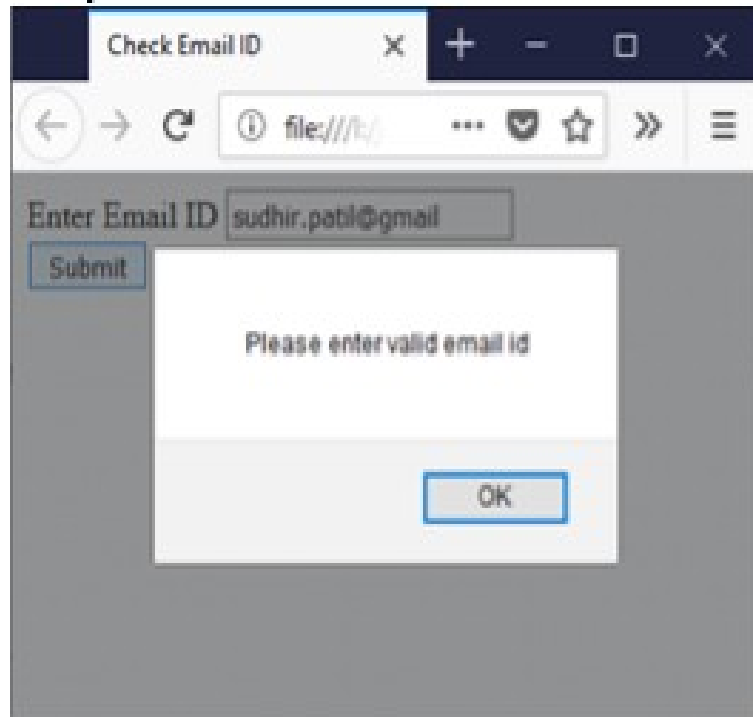


sample program

```
<html>
<head>
  <title>Check Email ID</title>
  <script>
    function checkEmail()
    {
      var email = document.getElementById('email').value
      var regex = /^[a-zA-Z0-9_\.]+@([a-zA-Z0-9_\.]+\.[a-zA-Z]{2,5})$/
      var res = regex.test(email)

      if(!res) {
        alert('Please enter valid email id')
      }
      else {
        alert('Thank you')
      }
    }
  </script>
</head>
<body>
  <form name="myform" action="#" method="post">
    Enter Email ID <input type="text" id="email" /><br/>
    <input type="button" value="Submit" onclick="checkEmail()" />
  </form>
</body>
</html>
```

Sample program output



Sample program

```
<script type="text/javascript">
function matchpass(){
var firstpassword=document.f1.password.value;
var secondpassword=document.f1.password2.value;

if(firstpassword==secondpassword){
return true;
}
else{
alert("password must be same!");
return false;
}
}
</script>

<form name="f1" action="register.jsp" onsubmit="return matchpass()">
Password: <input type="password" name="password" /> <br/>
Re-enter Password: <input type="password" name="password2" /> <br/>
<input type="submit">
</form>
```

Practical Exercises

1. Write a JavaScript program to check whether a string starts with 'Java' and false otherwise?
2. Write a JavaScript program to check whether a string "Script" presents at 5th (index 4) position in a given string, if "Script" presents in the string return the string without "Script" otherwise return the original one.
3. Write a JavaScript program to capitalize the first letter of each word of a given string. “the quick black fox “

Reading Resources/Materials

Links

- ✓ Paul Deitel,
- ✓ javaT point,
- ✓ rahultamkhane.medium.com,
- ✓ developer.mozilla.org

Thank You For Your Attention!!

Any Questions

